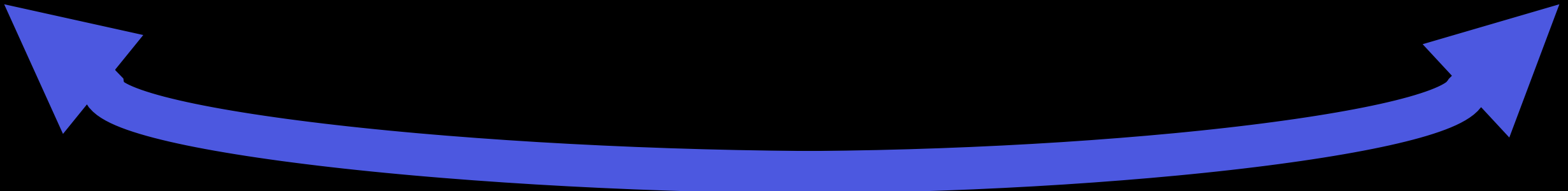


Alpha – Beta Pruning

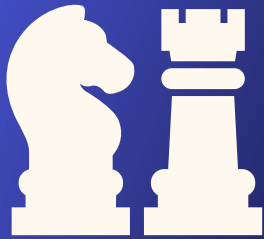
Alpha – Beta Pruning

is an optimization technique for the minimax algorithm that reduces the number of nodes evaluated in a game tree.

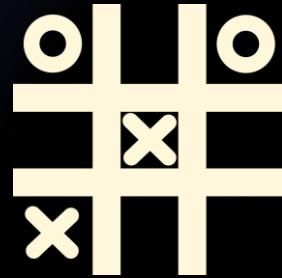


Alpha – Beta Pruning

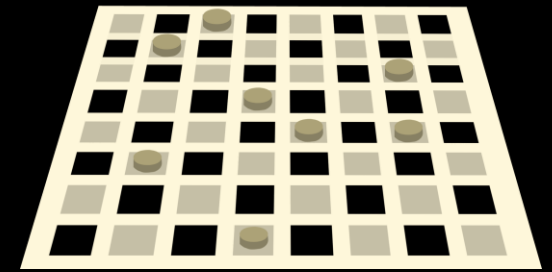
Use cases



Evaluating potential moves to maximize chances of winning while minimizing the opponent's options.

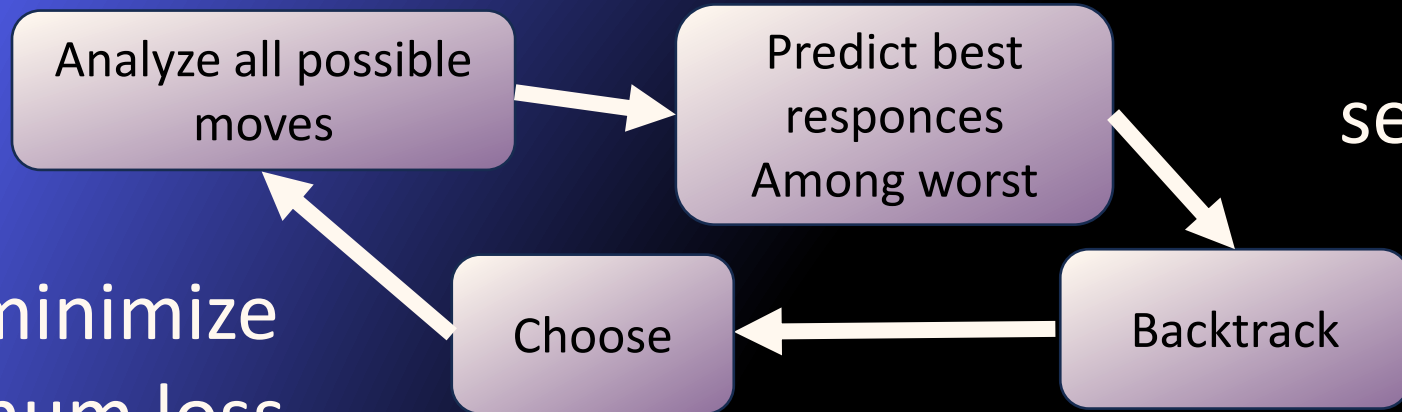


Quickly determining the best moves and blocking the opponent's winning paths.



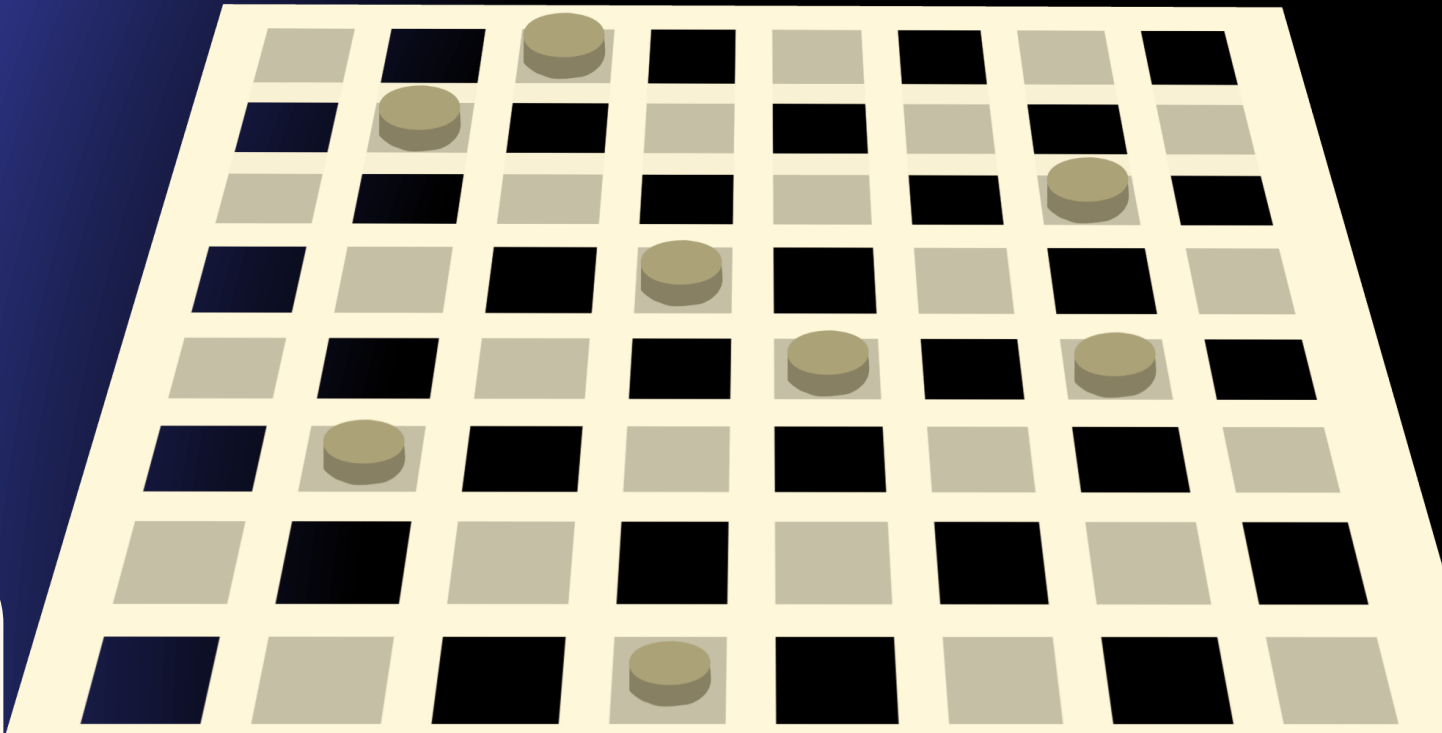
Finding optimal strategies for both players through an extensive game tree.

MiniMax Principle



seeks to maximize their gain

Seeks to minimize the maximum loss



Min

Max

MiniMax Principle

Analyze all possible moves

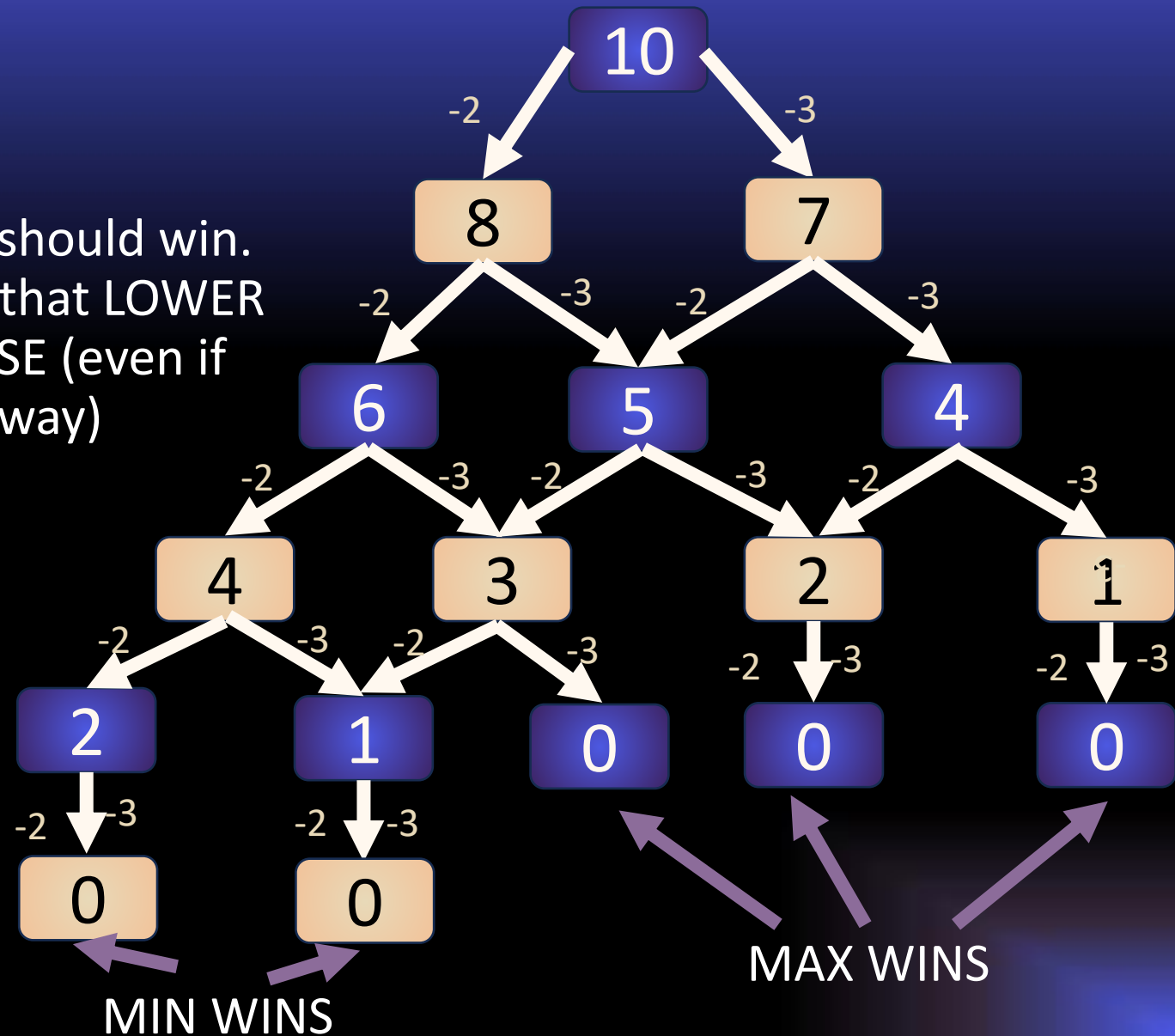
Predict best responses among worst

Choose

Backtrack

Player MIN starts and should win.
MIN should take steps that LOWER their chances to LOOSE (even if MIN loses anyway)

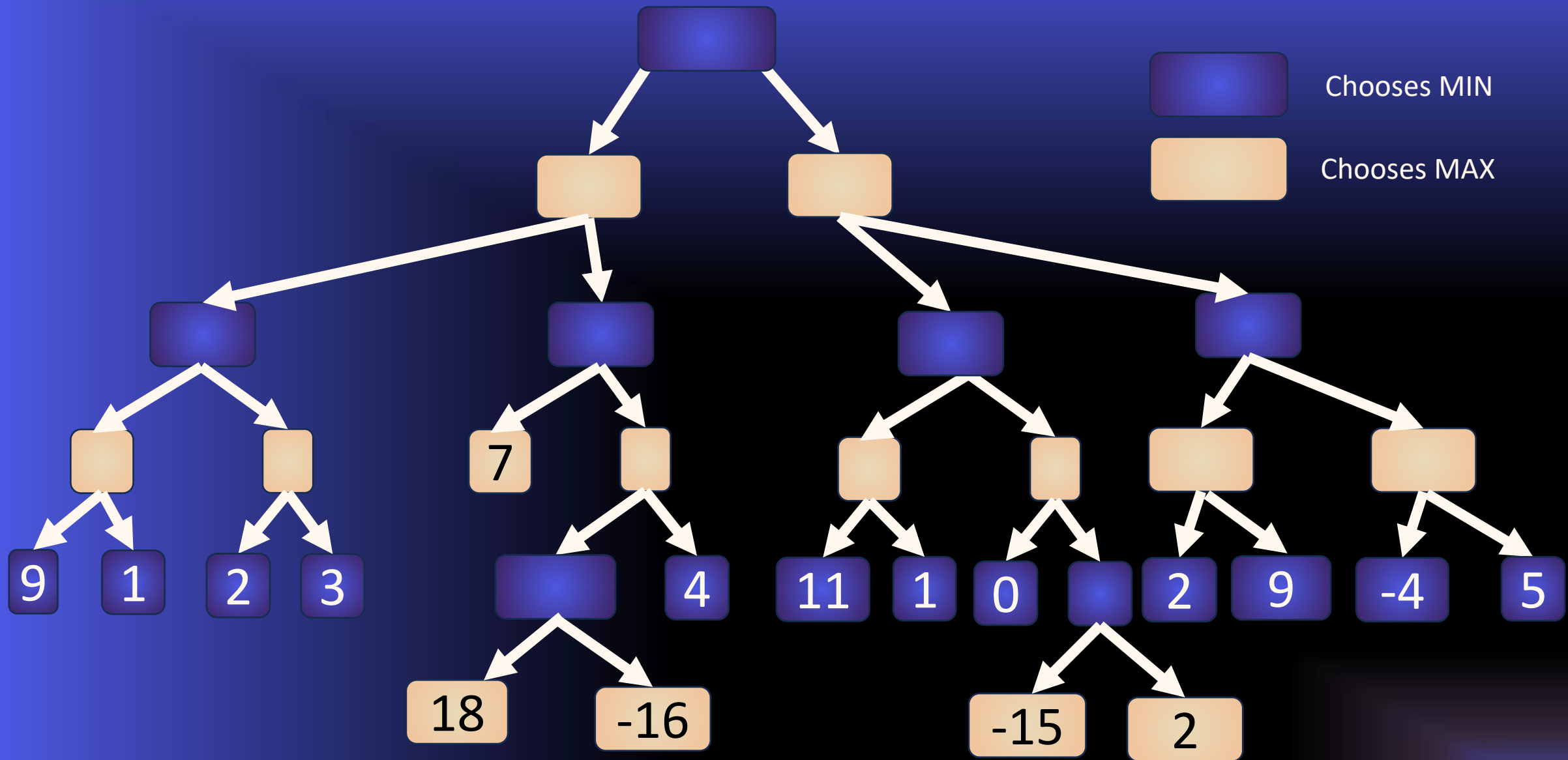
Player MAX wants to win and will take steps that increases their chances to win

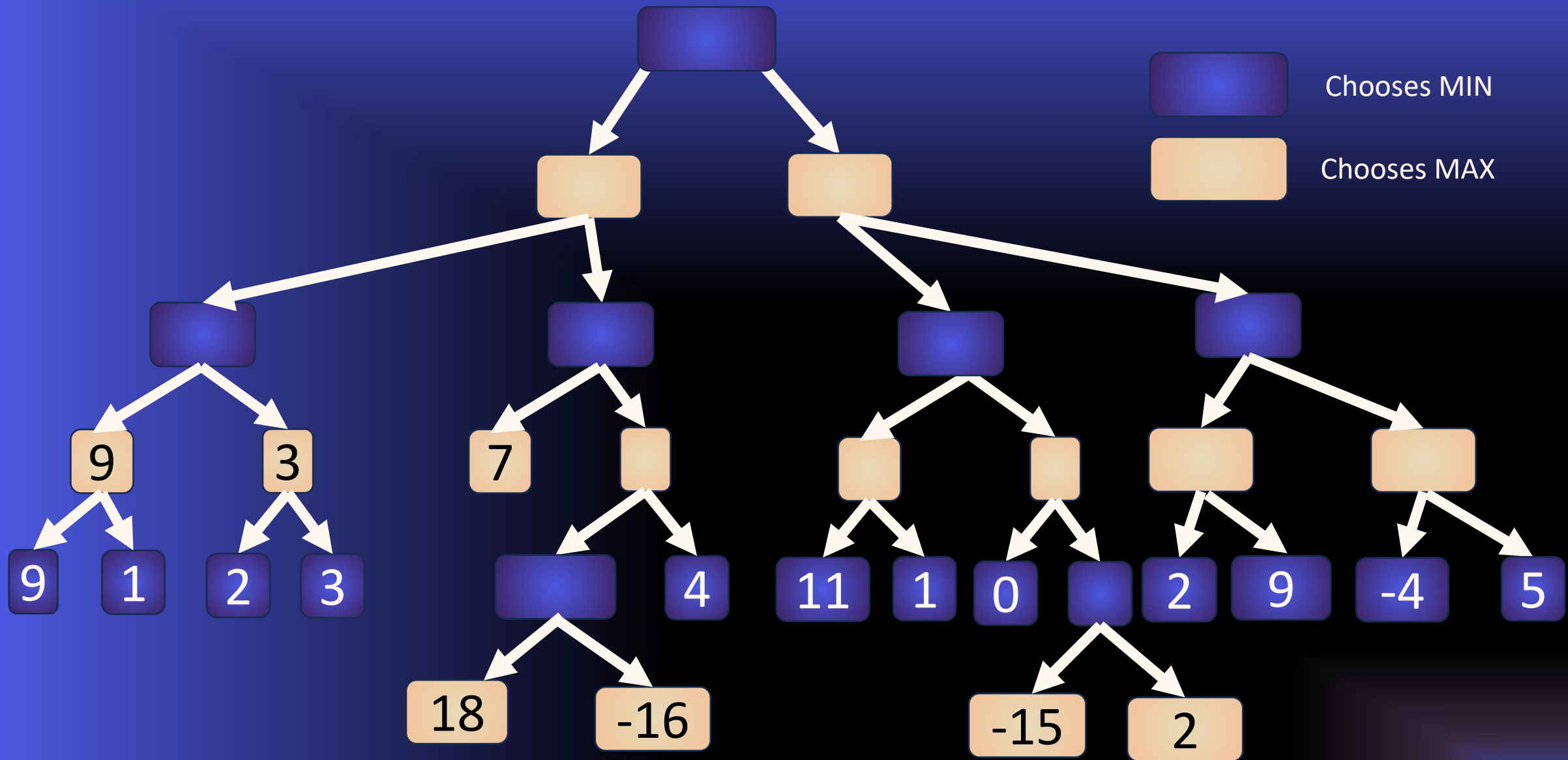


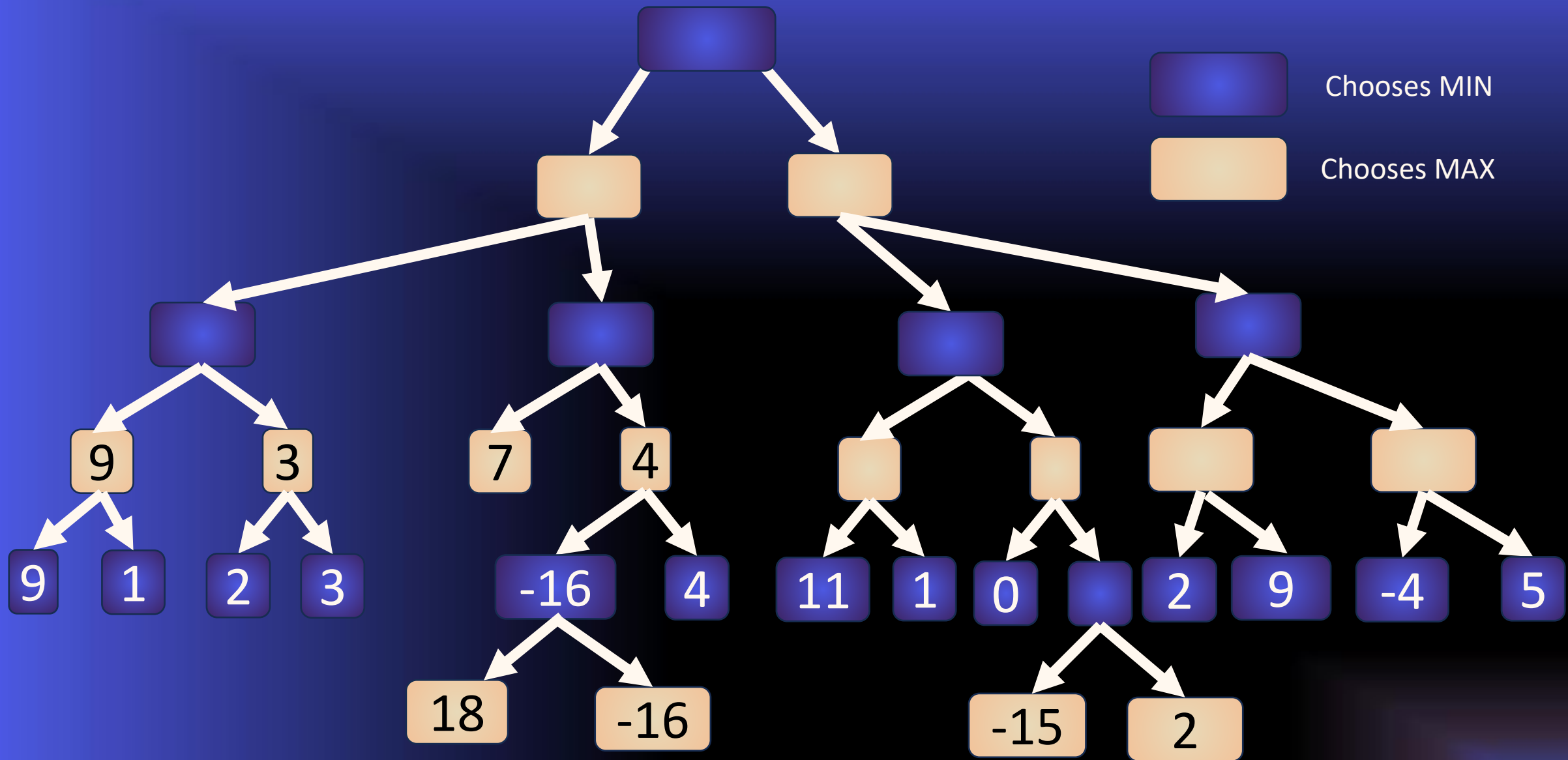
```
function minimax(position, depth, maximizingPlayer)
    if depth == 0 or game over in position
        return static evaluation of position Base of recursion

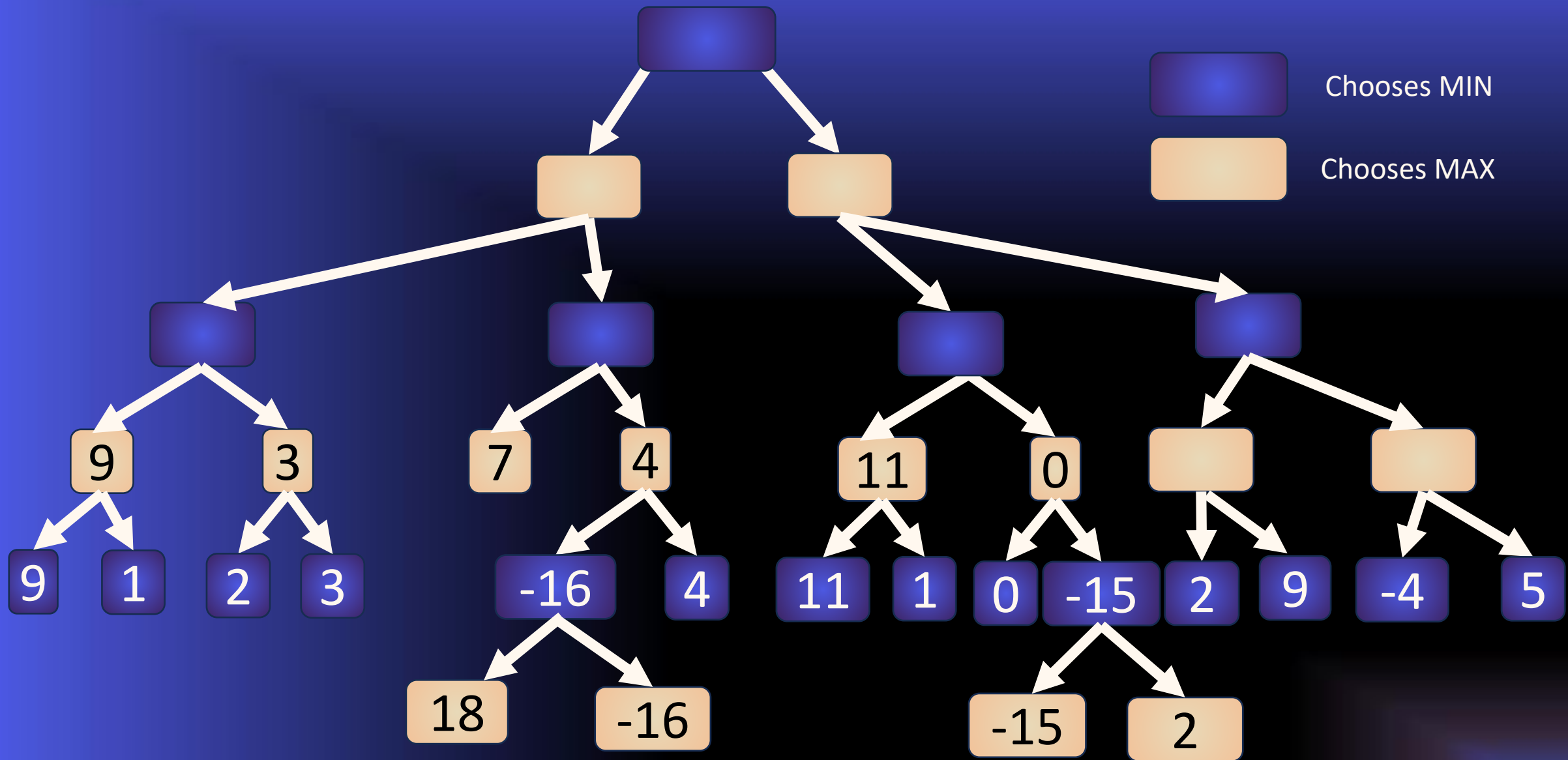
    if maximizingPlayer MAX chooses
        maxEval = -infinity
        for each child of position
            eval = minimax(child, depth - 1, false) MIN chooses
            maxEval = max(maxEval, eval)
        return maxEval

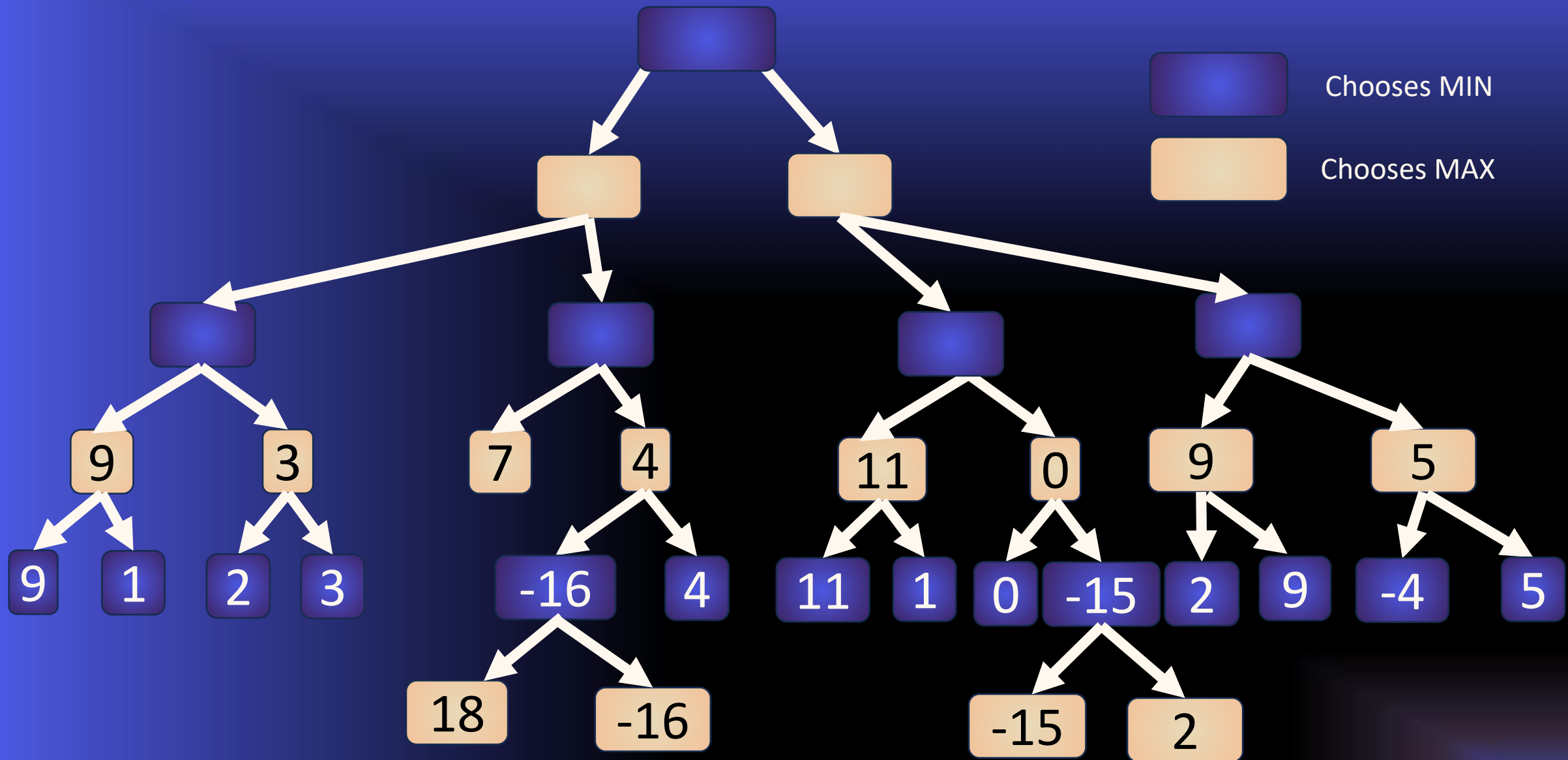
    else MIN chooses
        minEval = +infinity
        for each child of position
            eval = minimax(child, depth - 1, true) MAX chooses
            minEval = min(minEval, eval)
```

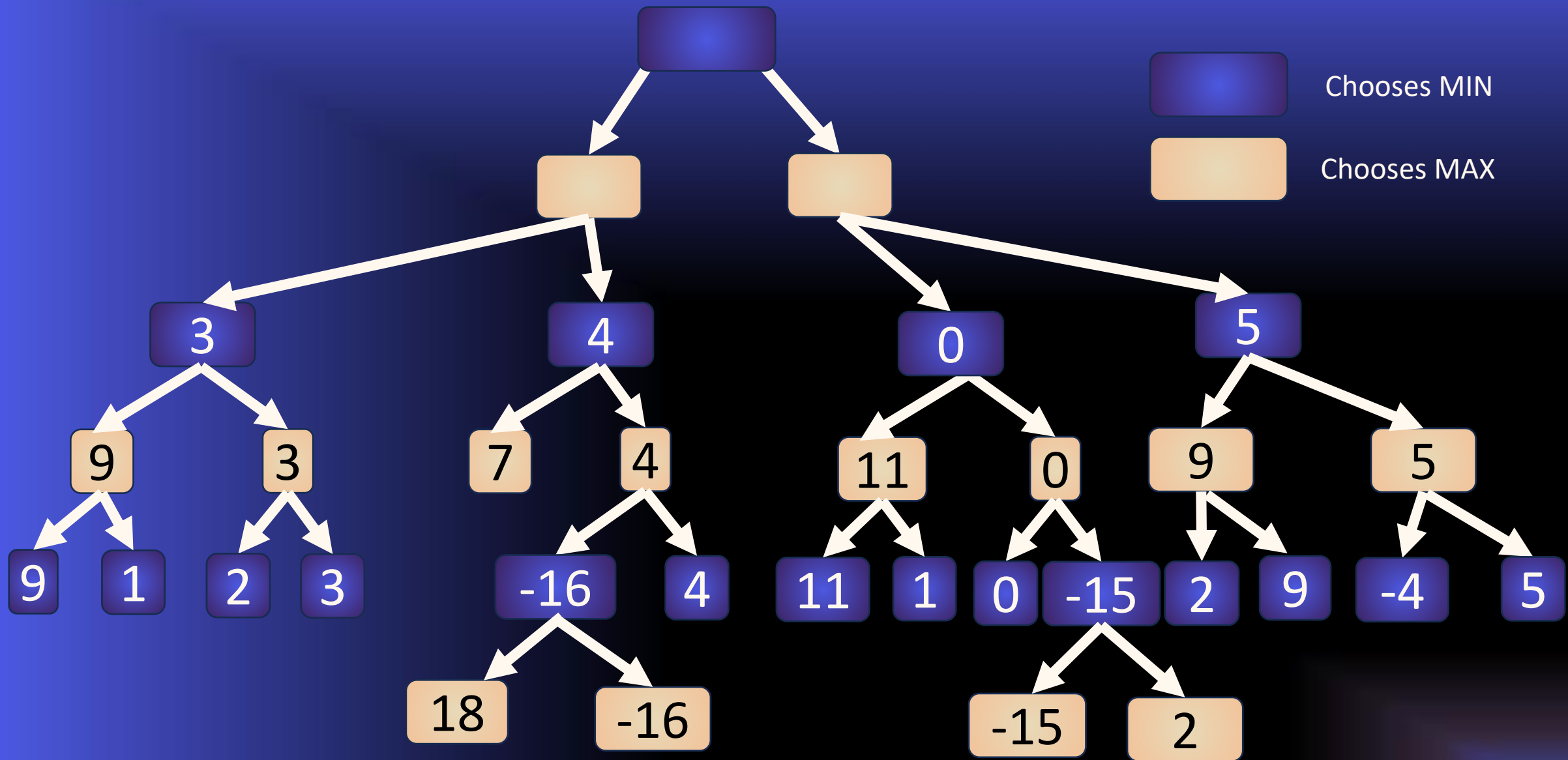


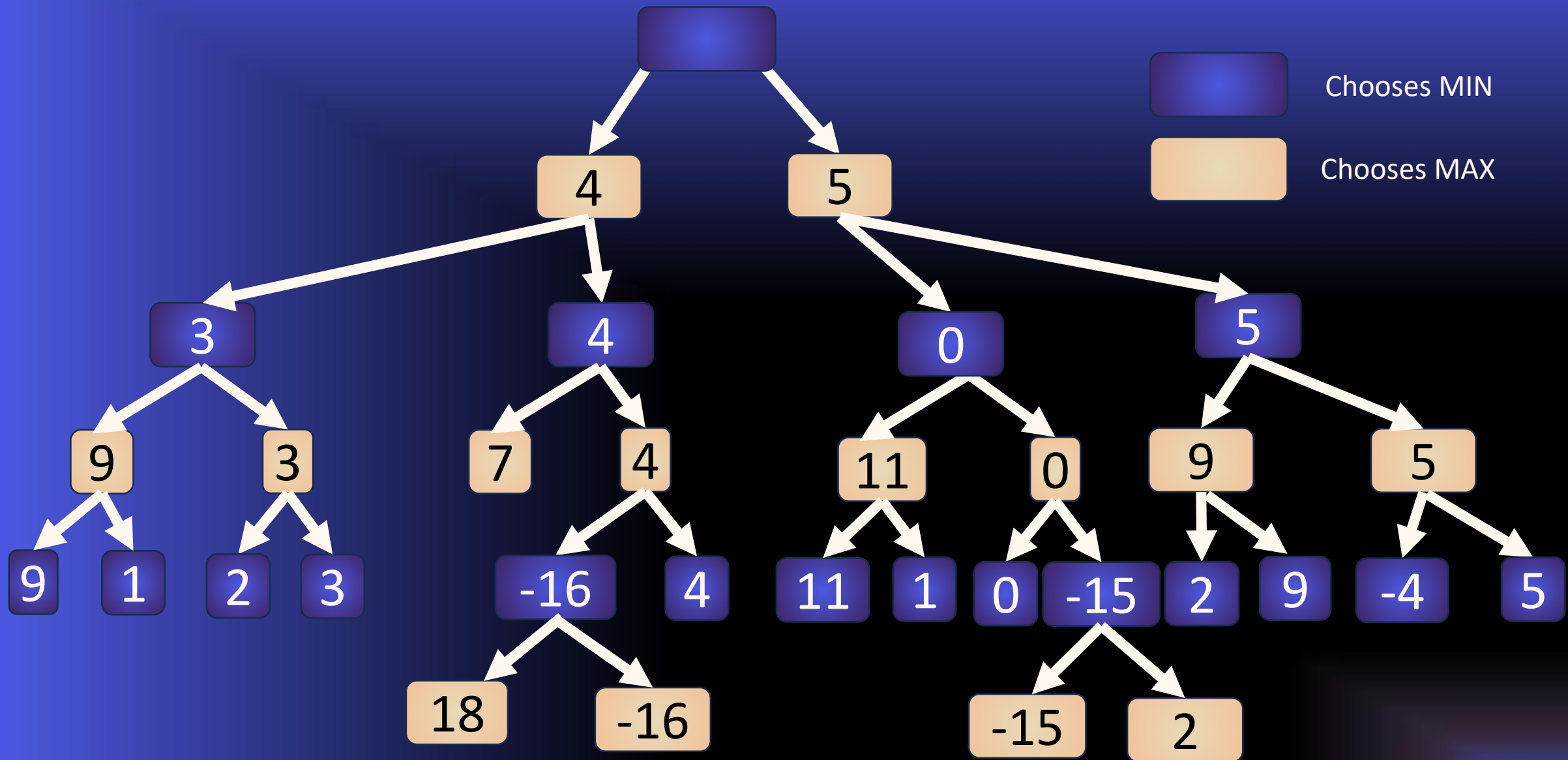


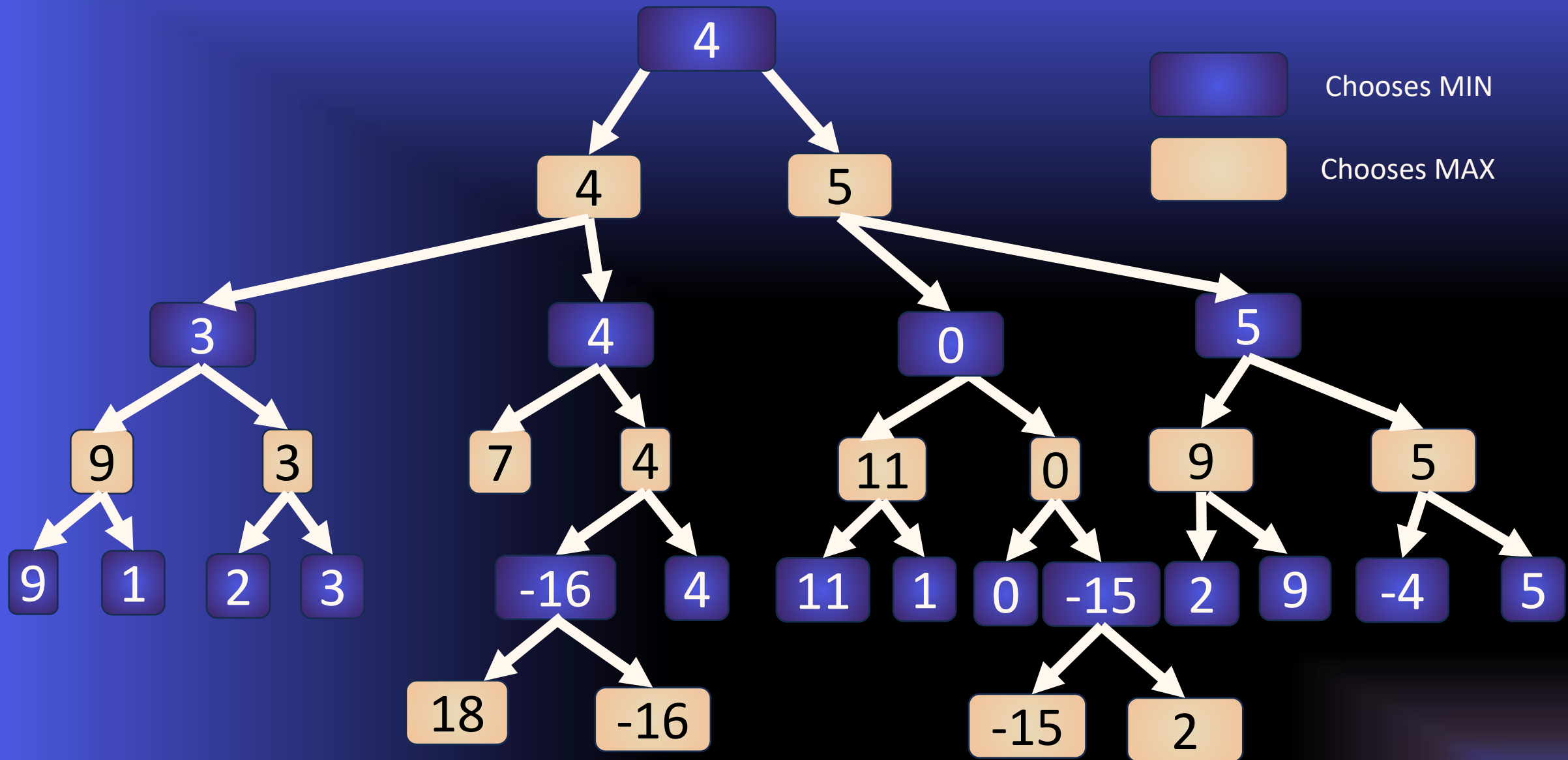












Alpha – Beta Pruning

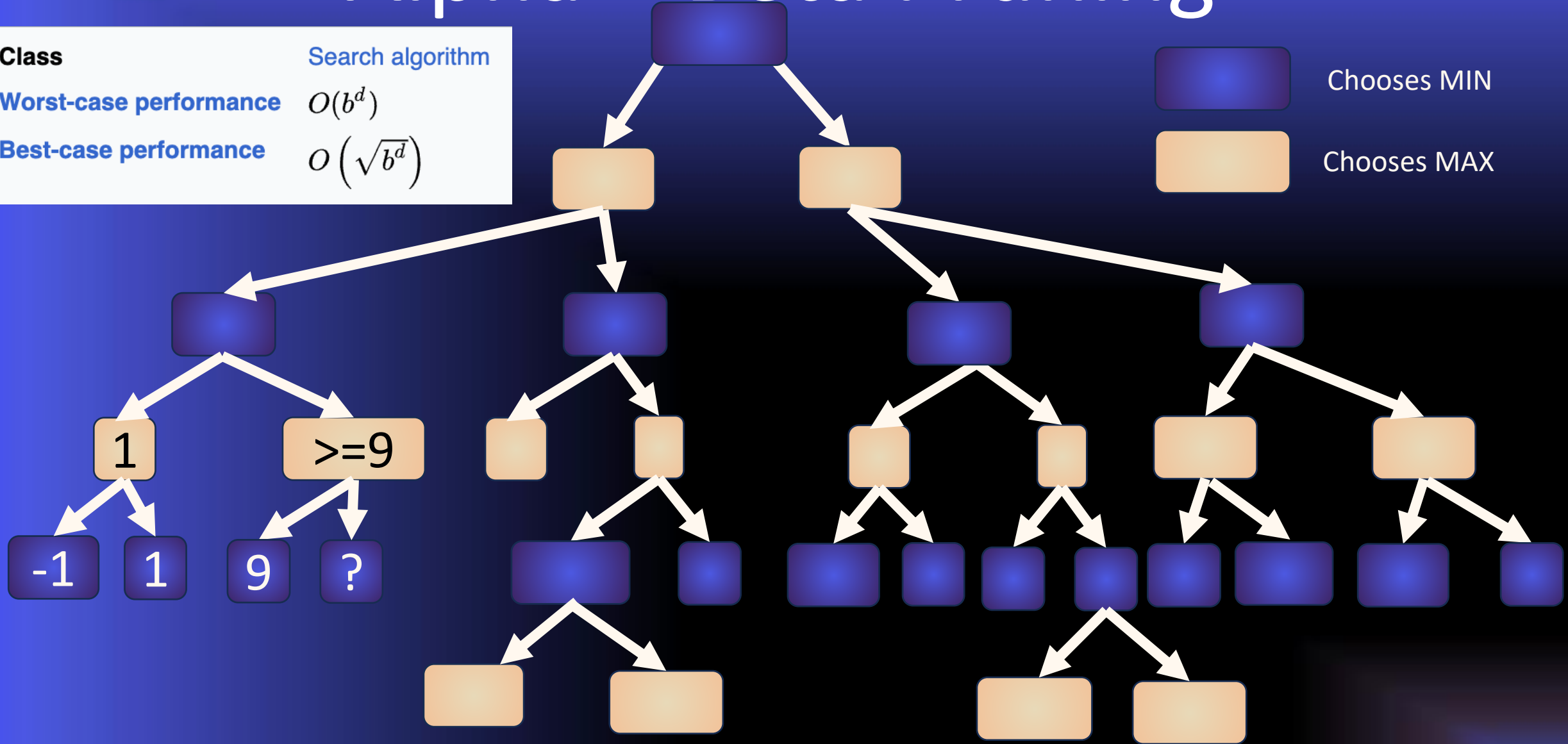
is an optimization technique for the minimax algorithm that reduces the number of nodes evaluated in a game tree allowing the algorithm to run more efficiently without affecting the outcome.

Alpha – Beta Pruning

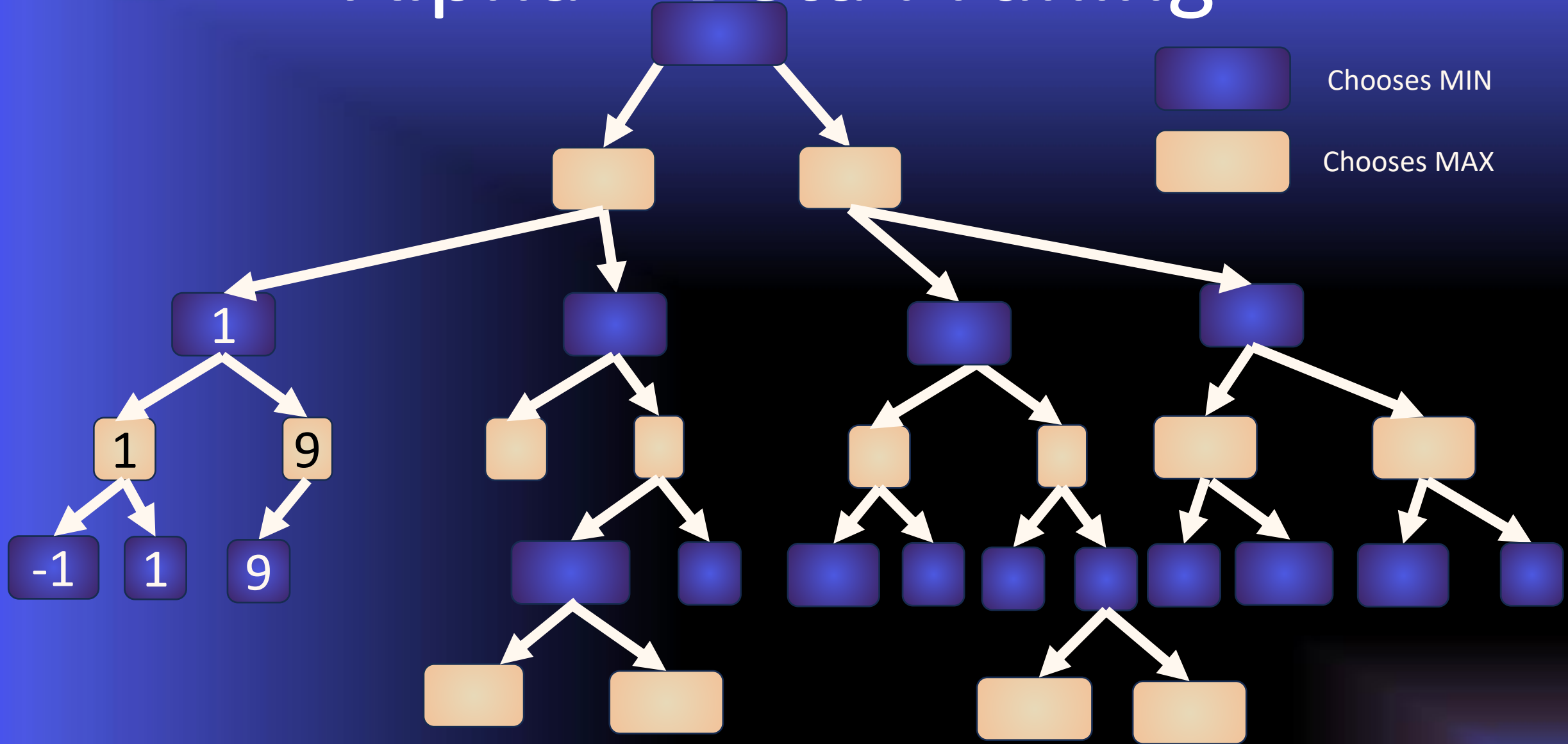
Class	Search algorithm
Worst-case performance	$O(b^d)$
Best-case performance	$O(\sqrt{b^d})$

Chooses MIN

Chooses MAX



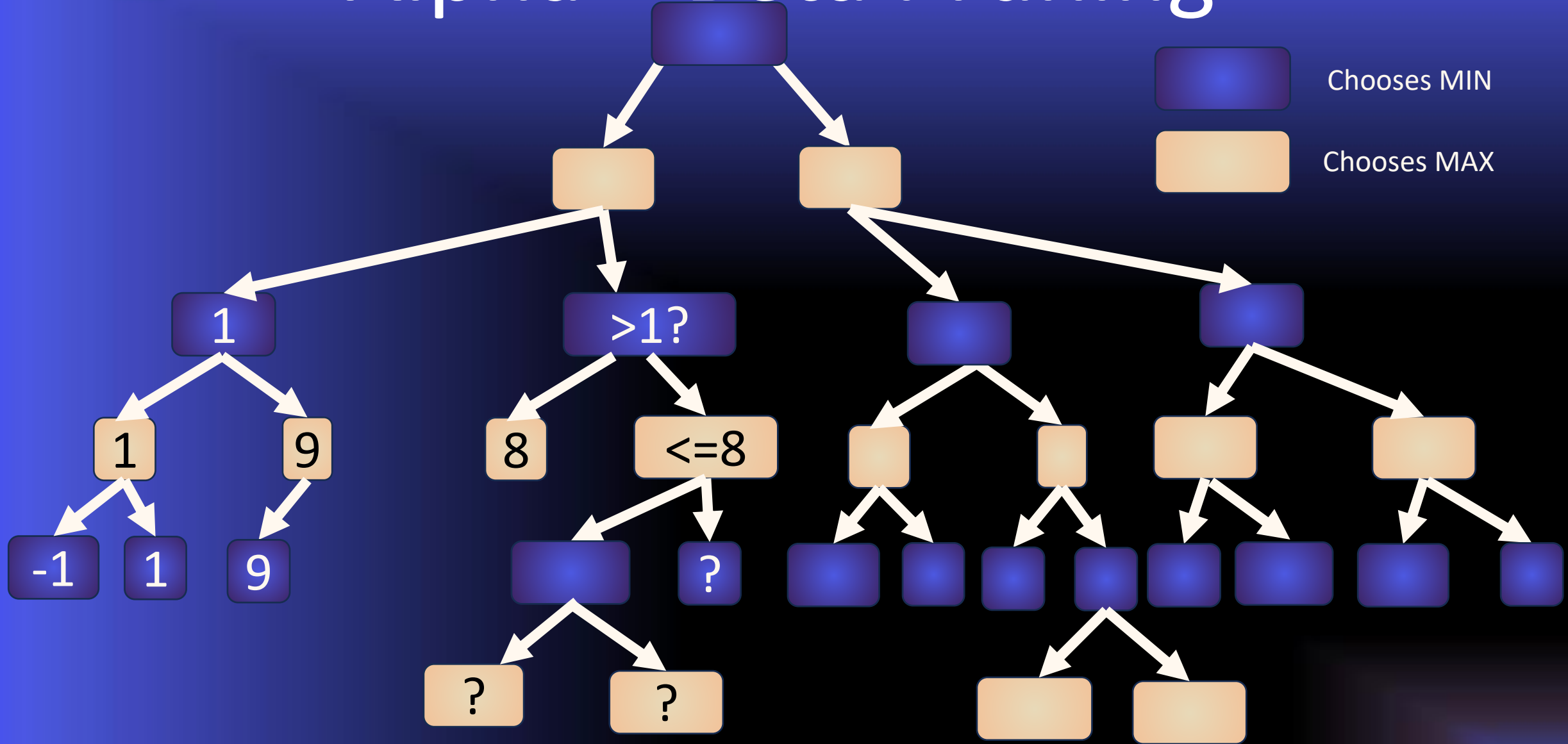
Alpha – Beta Pruning



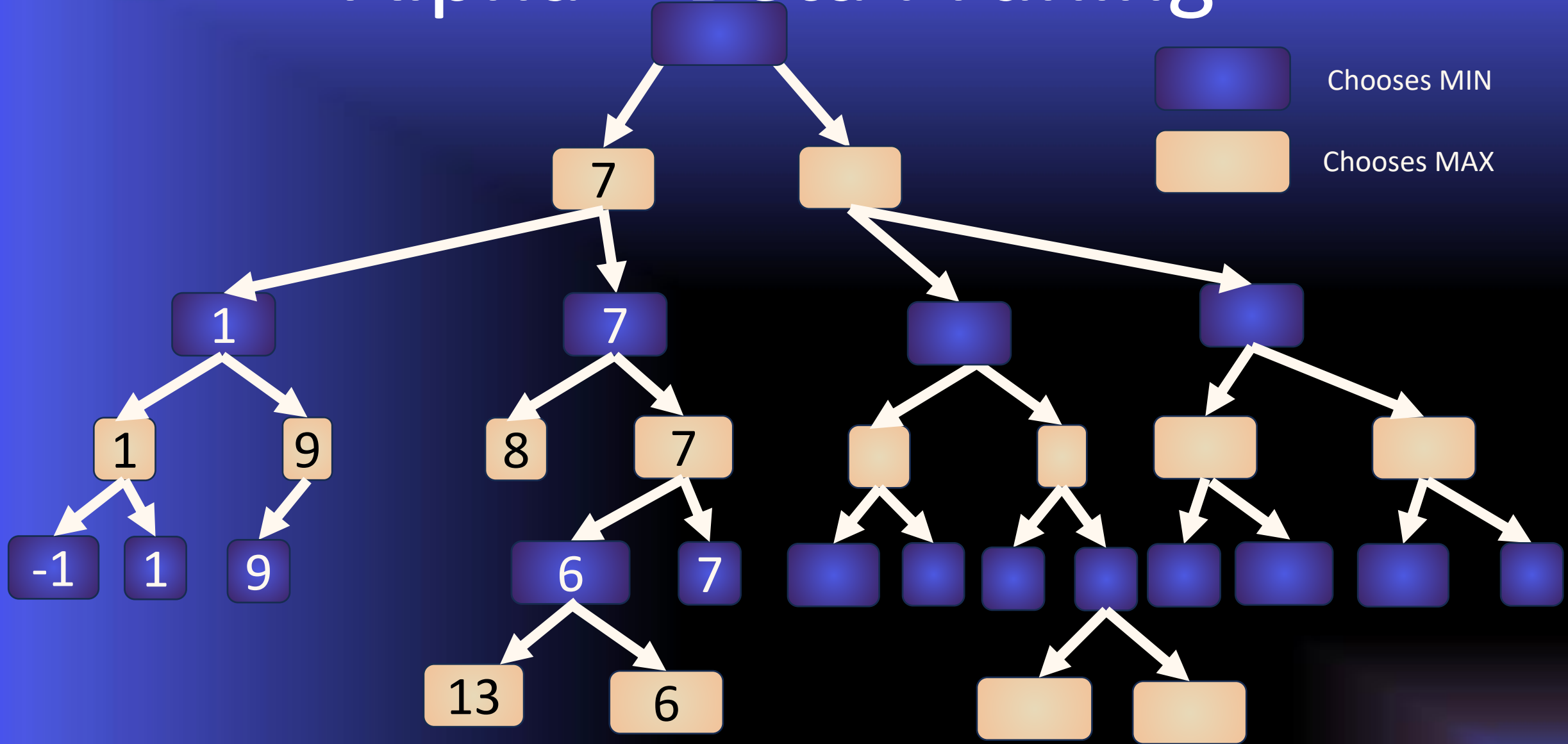
Chooses MIN

Chooses MAX

Alpha – Beta Pruning

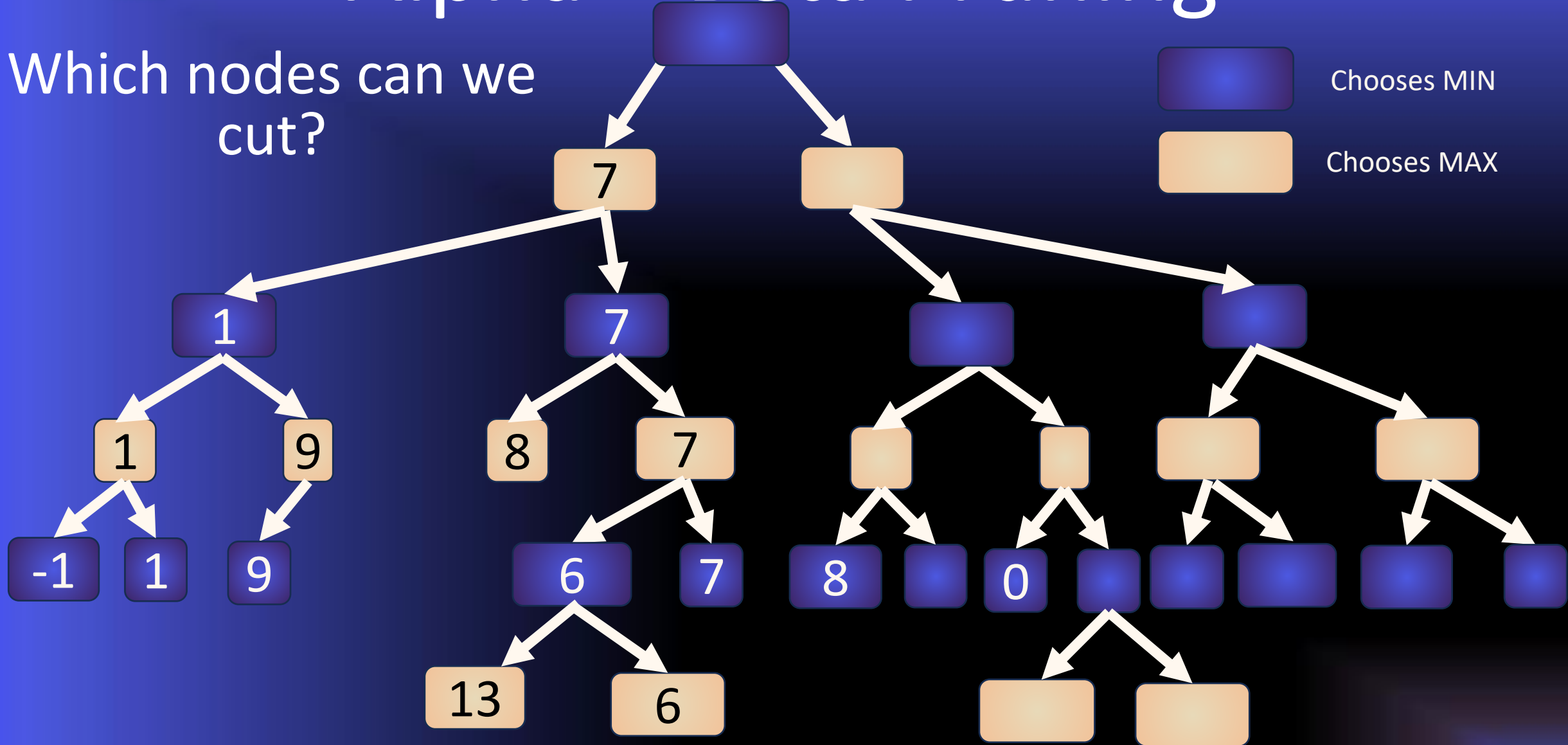


Alpha – Beta Pruning



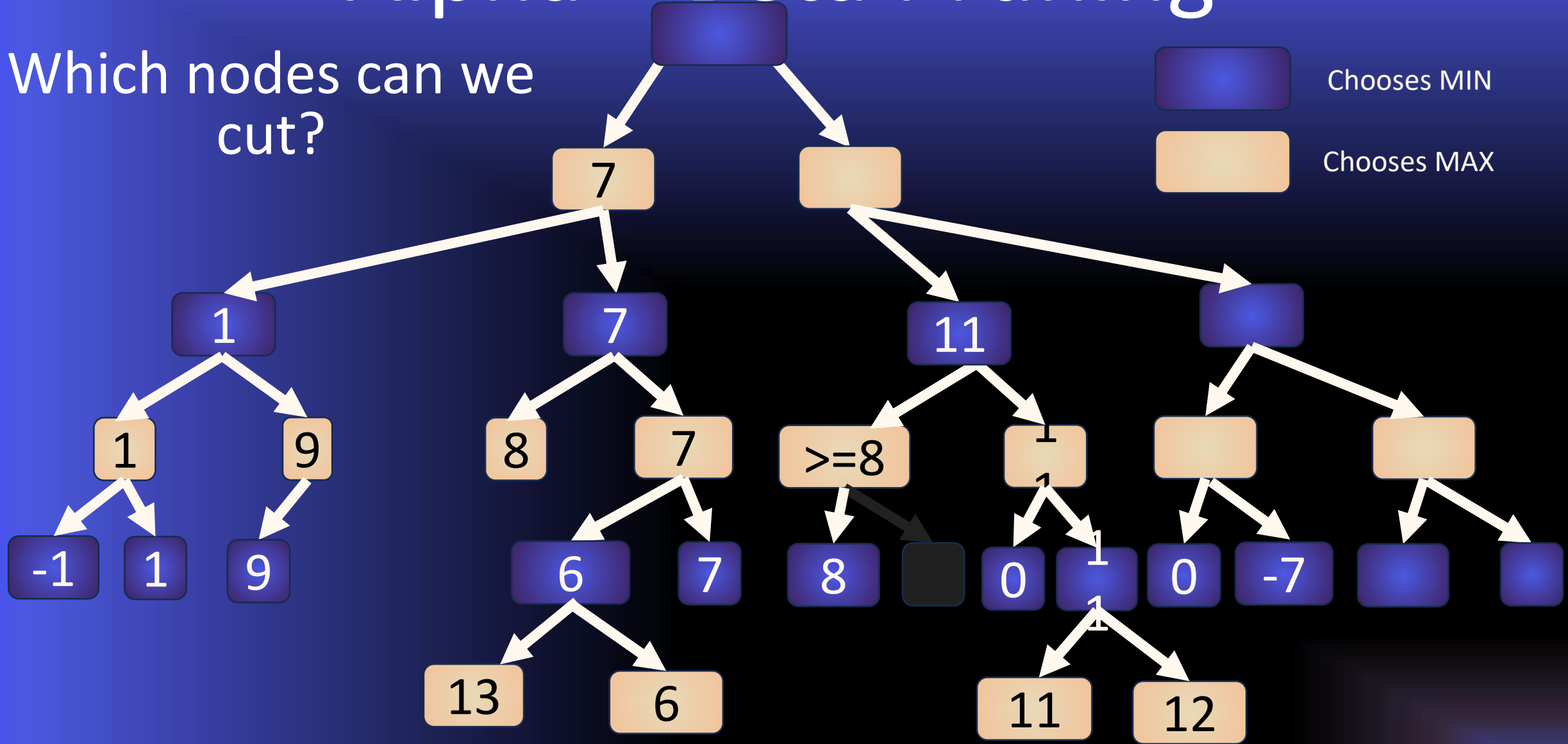
Alpha – Beta Pruning

Which nodes can we cut?



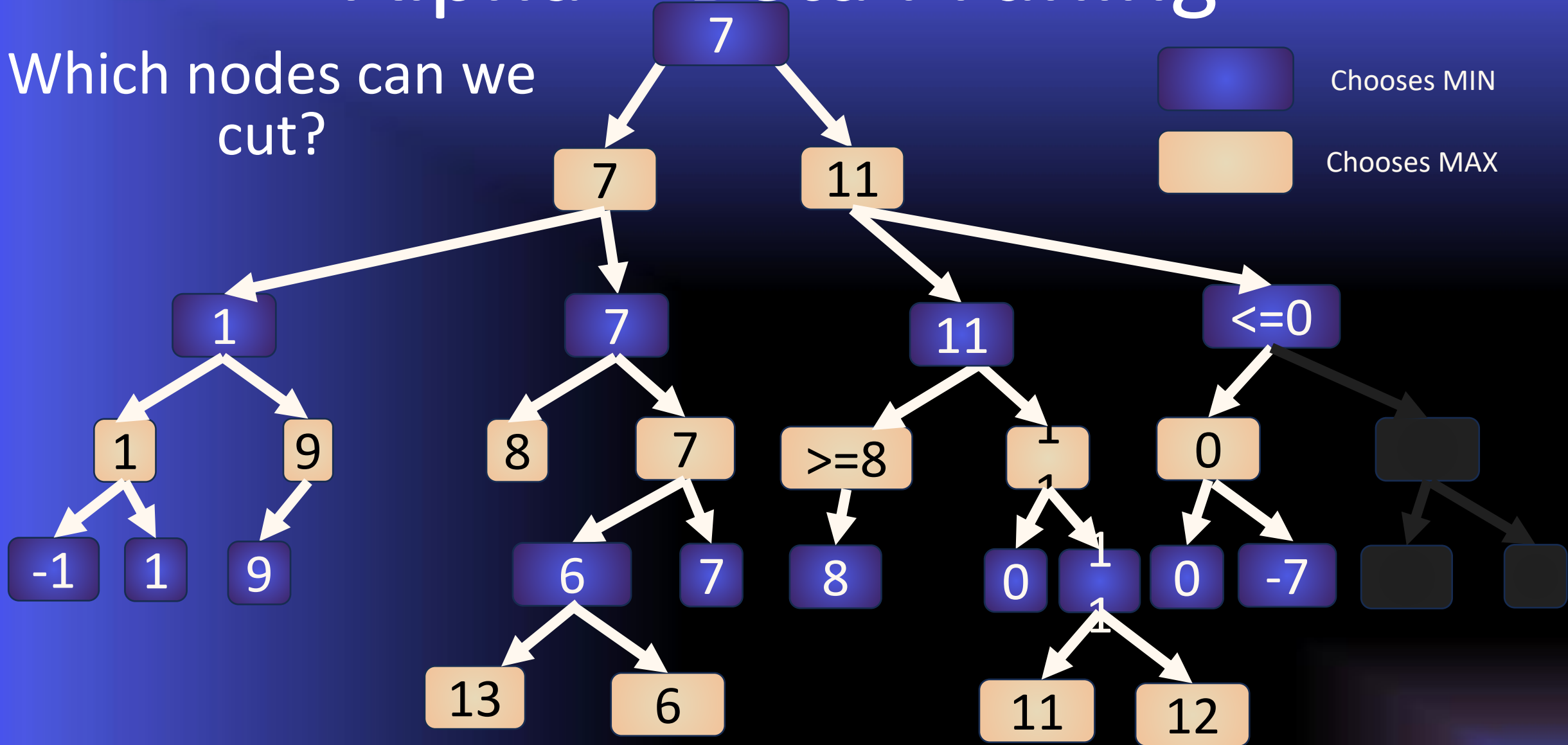
Alpha – Beta Pruning

Which nodes can we cut?



Alpha – Beta Pruning

Which nodes can we cut?



Alpha – Beta Pruning

```
function minimax(position, depth, alpha, beta, maximizingPlayer)
    if depth == 0 or game over in position
        return static evaluation of position

    if maximizingPlayer
        maxEval = -infinity
        for each child of position
            eval = minimax(child, depth - 1, alpha, beta, false)
            maxEval = max(maxEval, eval)
            alpha = max(alpha, eval)
            if beta <= alpha
                break
        return maxEval

    else
        minEval = +infinity
        for each child of position
            eval = minimax(child, depth - 1, alpha, beta, true)
            minEval = min(minEval, eval)
            beta = min(beta, eval)
        return minEval
```

Pruning
condition

Alpha - The best value that the maximizing player can guarantee so far. It represents the lower bound for the maximizing player.

Beta - The best value that the minimizing player can guarantee so far. It represents the upper bound for the minimizing player.

Thank you for attention!

Group:
Koltyshev Egor
Akimov Ilya